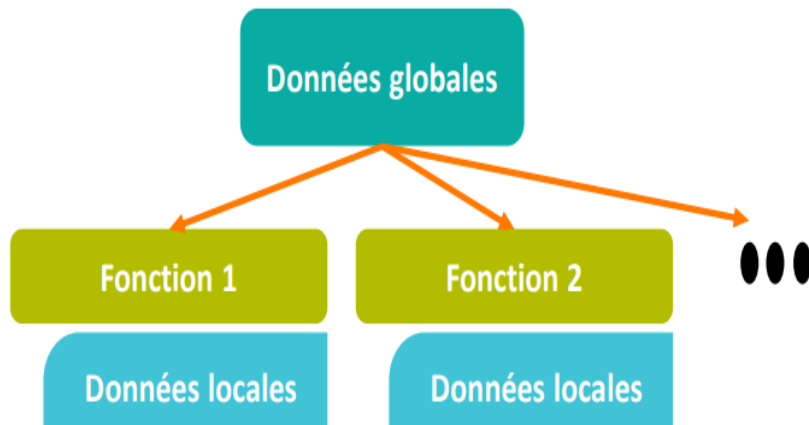


l'orientée objet en PHP

Intérêt de programmer en Orienté Objet en PHP

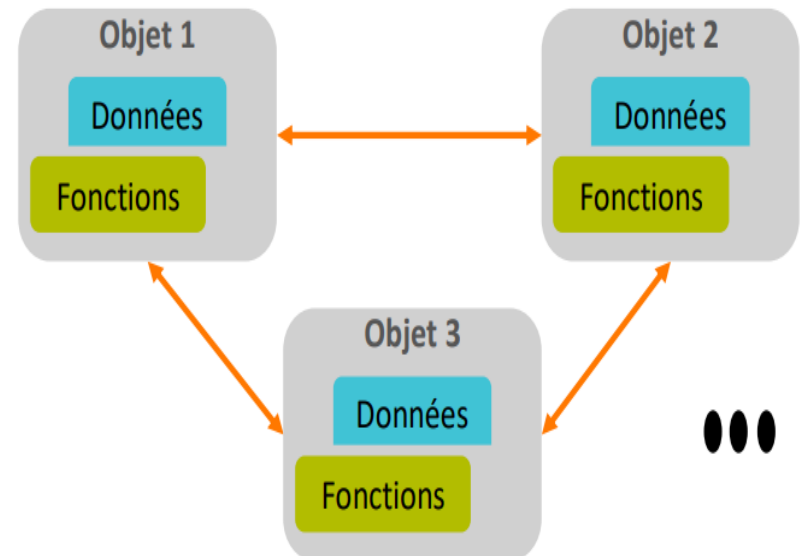
Programmation procédurale

- Le programme est divisé en procédures ou fonctions.
- Chaque fonction contient des données différentes.
- Une procédure permet d'effectuer des opérations sur les données généralement contenues dans des variables.
- Les opérations sont exécutées selon leur ordre d'écriture dans le script.



Programmation orientée objet

- Le programme est divisé en objets.
- Les objets agissent comme une seule unité.
- Un objet est une entité qui va pouvoir contenir un ensemble de fonctions et de variables.
- Regrouper des tâches précises au sein d'objets pour obtenir une nouvelle organisation du code



Avantages d'une programmation orientée objet (POO)

Meilleure organisation du projet.

Facilite la maintenance du code.

Souplesse à évoluer le logiciel.

Factorisation des comportements, puisque les fonctions deviennent interdépendantes.

Masquage des données ou l'encapsulation consiste à masquer les détails d'implémentation d'un objet, en définissant une interface.

Spécificateur d'accès en utilisant un accesseur qui est une méthode d'accès pour connaître ou modifier la valeur d'un attribut d'un objet.

Transmission des propriétés grâce à l'héritage (ainsi que le polymorphisme), ainsi éviter la duplication de code.

l'agrégation qui permet de définir des objets composés d'autres objets.

Les objets et les classes

Une classe :

- Est un ensemble de variables et de fonctions (attributs et méthodes).
- Contient le nom des propriétés qu'on pourra manipuler
- Les propriétés, ce sont des variables internes où on stocke des valeurs.
- Les méthodes sont des fonctions internes à la classe.
- Un objet est une instance de la classe pour pouvoir l'utiliser.

Exemple :

- **Classe** : Voiture
- **Propriétés** : Type, Energie, Puissance réelle, Boite de vitesse, Portes/Places , Vitesse maximale, Consommation moyenne, Emission de CO2, Couleur ...
- **Méthodes** : Accélérer, Freiner, Klaxonner, Feux, Tourner à droite, Reculer ...
- **Objets** : voiture rouge type hybride diesel 4 portes ... , voiture bleu essence 8cv type berline ..., ...

Syntaxe de base

- Une définition de classe basique commence par le mot-clé `class`, suivi du nom de la classe.

Le nom de la classe :

- Peut être `^[a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*$`
- Ne doit pas être un mot réservé en PHP

```
<?php
class NomClass
{
    // Déclaration d'une propriété

    // Déclaration des méthodes
}
?>
```

La pseudo-variable `$this` est :

- Disponible lorsqu'une méthode est appelée depuis un contexte objet.
- La valeur de l'objet appelant.

- Le mot-clé `new` est utilisé pour créer une instance d'une classe.
- Le mot-clé `extends` permet d'étendre les constantes, méthodes et les propriétés à une autre class.

```
<?php
class Voiture
{
    // Déclaration d'une propriété

    // Déclaration des méthodes
}
?>
```

Déclaration des attributs

Types pour déclarer un attribut :

- **public** : l'attribut sera accessible de l'intérieur de la classe dont il est membre comme de l'extérieur.
 - **private** : dans ce cas, l'attribut est accessible seulement de l'intérieur de la classe dont il est membre.
 - **protected** : dans ce cas, l'attribut est accessible seulement de l'intérieur de la classe dont il est membre ainsi que des classes qui héritent de cette classe.
- L'encapsulation est un principe fondamental de la POO. Il vise à masquer les attributs aux utilisateurs du code. C'est la visibilité `private` ou `protected` qui est recommandée.

```
class Voiture
{
    private $type = "Berline";
    private $Vitesse = 0;
    // Déclaration des méthodes
}
```

Déclaration des méthodes

Types pour déclarer une méthode :

- **public** : la méthode sera accessible de l'intérieur de la classe dont il est membre comme de l'extérieur.
- **private** : dans ce cas, la méthode est accessible seulement de l'intérieur de la classe. Généralement sollicitée par une autre méthode `public`.

```
class Voiture
{
    // Déclaration des méthodes

    public function accelerer(){
        // Corps de la méthode accelerer()
    }

    public function freiner(){
        // Corps de la méthode freiner()
    }
}
```

Les fonctionnalités d'un objet

- Le mot-clé `extends` permet de définir une classe dérivée.
- Le mot-clé `function` permet de déclarer une méthode.
- Le mot-clé `parent` permet d'accéder au parent de l'objet courant.
- `$this->` est obligatoire pour accéder aux membres de l'objet courant.
- Mot-clé `final` empêche les classes enfants de redéfinir une méthode ou constante en préfixant la définition avec `final`. Si la classe elle-même est définie comme finale, elle ne pourra pas être étendue.
- Une propriété peut être déclarée avec le modificateur `readonly`, qui empêche la modification de la propriété après l'initialisation.

Fonctions Classes / Objets

__autoload

Tente de charger une classe indéfinie

class_alias

Crée un alias de classe

class_exists

Vérifie si une classe a été définie

enum_exists

Checks if the enum has been defined

get_called_class

Le nom de la classe en "Late Static Binding"

get_class_methods

Retourne les noms des méthodes d'une classe

get_class_vars

Retourne les valeurs par défaut des propriétés d'une classe

get_class

Retourne le nom de la classe d'un objet

get_declared_classes

Liste toutes les classes définies dans PHP

get_declared_interfaces

Retourne un tableau avec toutes les interfaces déclarées

get_declared_traits

Retourne un tableau contenant tous les traits déclarés

get_mangled_object_vars

Returns an array of mangled object properties

get_object_vars

Retourne les propriétés d'un objet

get_parent_class

Retourne le nom de la classe parente d'un objet

interface_exists

Vérifie si une interface a été définie

is_a

Vérifie si l'objet est une instance d'une classe donnée ou a cette classe parmi ses parents

is_subclass_of

Détermine si un objet est une sous-classe d'une classe donnée ou l'implémente

method_exists

Vérifie si la méthode existe pour une classe

property_exists

Vérifie si un objet ou une classe possède une propriété

trait_exists

Vérifie si un trait existe

Utilisation des méthodes magiques en PHP

Les méthodes spéciales peuvent écraser l'action par défaut de PHP.

Les méthodes commençant par double sous-tirets `__` sont réservées par PHP. (Cette notation est réservée aux méthodes magiques)

17 méthodes sont considérées magiques

Toutes les méthodes magiques, à l'exception de `__construct()`, `__destruct()`, et `__clone()`, doivent être déclarées en tant que public, sinon une `E_WARNING` est émise.

__construct()

Syntaxe : __construct(mixed ...\$values = ""): void

Les classes qui possèdent une méthode constructeur appellent cette méthode à chaque création d'une nouvelle instance de l'objet. Si vous voulez utiliser un constructeur parent, il sera nécessaire de faire appel à parent::__construct() depuis le constructeur enfant. Un constructeur peut définir un nombre d'arguments, qui peuvent être requis, avoir un type, et peuvent avoir une valeur par défaut.

```
class Voiture{
    public function __construct(){
        $this->couleur="Blanche";
        $this->vitesse=0;
    }
}
$maVoiture = new Voiture(); // le constructeur est appelé et les
attributs initialisés
```

__destruct()

Syntaxe : __destruct(): void

La méthode destructeur est appelée dès qu'il n'y a plus de référence sur un objet donné.

Si vous voulez utiliser un destructeur parent, il sera nécessaire de faire appel à parent::__destruct() depuis le destructeur enfant.

Le destructeur sera appelé même si l'exécution du script est stoppée en utilisant la fonction exit().

Surcharge de propriétés

- `__set()` :
 - Est sollicitée lors de l'écriture de données vers des propriétés inaccessibles (protégées ou privées) ou non existante.
 - Syntaxe : `public __set(string $name, mixed $value): void`
- `__get()` :
 - Est appelée pour lire des données depuis des propriétés inaccessibles (protégées ou privées) ou non existante.
 - Syntaxe : `public __get(string $name): mixed`
- `__isset()` :
 - Est sollicitée lorsque `isset()` ou `empty()` sont appelées sur des propriétés inaccessibles (protégées ou privées) ou non existante.
 - Syntaxe : `public __isset(string $name): bool`
- `__unset()` :
 - Est invoquée lorsque `unset()` est appelée sur des propriétés inaccessibles (protégées ou privées) ou non existante.
 - Syntaxe : `public __unset(string $name): void`

__sleep() et __serialize()

- **__sleep()** :
 - Est de valider des données en attente ou d'effectuer des opérations de nettoyage.
 - Syntaxe : `public __sleep(): array`
- **__serialize()**:
 - Vérifie si la classe a une méthode avec le nom magique `__sleep()`.
 - Retourne les noms de propriétés privées des classes parentes.
Construit et retourne un tableau associatif de paire clé/valeur qui représente la forme linéarisée de l'objet
 - Syntaxe : `public __serialize(): array`

Si `__serialize()` et `__sleep()` sont tout les deux définie dans le même objet, alors seulement `__serialize()` sera appelé.

__wakeup() et __unserialize():

- **__wakeup()** :
 - Est de rétablir toute connexion de base de données qui aurait été perdue durant la linéarisation et d'effectuer des tâches de réinitialisation.
 - Syntaxe : `public __wakeup(): void`
- **__unserialize()**:
 - Vérifie si la classe a une méthode avec le nom magique `__wakeup()`.
 - Peut reconstruire toute ressource que l'objet pourrait posséder.
 - Syntaxe : `public __unserialize(array $data): void`

Si `__unserialize()` et `__wakeup()` sont tout les deux définie dans le même objet, alors seulement `__unserialize()` sera appelée.

Surcharge de méthodes

- `__call()` :
 - Est appelée lorsque l'on invoque des méthodes inaccessibles dans un contexte objet.
 - Syntaxe: `public __call(string $name, array $arguments): mixed`
- `__callStatic()` :
 - Est lancée lorsque l'on invoque des méthodes inaccessibles dans un contexte statique.
 - Syntaxe : `public static __callStatic(string $name, array $arguments): mixed`

Autres méthodes magiques

- **__toString():**
 - Détermine comment l'objet doit réagir lorsqu'il est traité comme une chaîne de caractères.
 - Syntaxe : `public __toString(): string`
- **__invoke() :**
 - Est appelée lorsqu'un script tente d'appeler un objet comme une fonction.
 - Syntaxe : `__invoke( ...$values): mixed`
- **__set_state() :**
 - Est appelée pour les classes exportées par la fonction `var_export()`.
 - Syntaxe : `static __set_state(array $properties): object`
- **__clone() :**
 - Un nouvel objet sera appelée, pour permettre à chaque propriété qui doit l'être d'être modifiée.
 - Syntaxe : `__clone(): void`